Capitolo 3

Requisiti, UML e analisi orientata agli oggetti

3.1 Requisiti, specifiche e feature

La progettazione parte dai bisogni del cliente. L'analisi deve trasformarli in servizi richiesti, vincoli e criteri con cui verificare il risultato.

In questo passaggio distinguiamo tre termini. Un **requisito** è ciascuna caratteristica che il software deve avere. Una **specifica** è la descrizione rigorosa di tali caratteristiche. Una **feature** raggruppa requisiti correlati che permettono all'utente di soddisfare un obiettivo.

I requisiti tendono a essere granulari, cioè numerosi e piccoli, e possono cambiare nel tempo. Rendere esplicito ciò che viene richiesto e come verificarlo permette di seguire questi cambiamenti senza perdere il collegamento con il bisogno iniziale.

UML, acronimo di *Unified Modeling Language*, è una notazione grafica standardizzata per rappresentare visivamente un sistema software. UML si usa sia per comunicare con il cliente, sia per ragionare sulla struttura del programma e sulle interazioni tra oggetti. I diagrammi trattati sono: casi d'uso, attività, stati, collaborazione, classi e sequenza.

L'*analisi dei requisiti* è la fase che porta a definire le specifiche e stabilisce i servizi richiesti e i vincoli del software. L'ingegneria dei requisiti comprende:

1. **studio di fattibilità**, nel quale si valuta se l'azienda possiede risorse e competenze per realizzare il sistema;
2. **analisi dei requisiti**, nella quale si chiariscono le richieste iniziali del cliente;
3. **specifica dei requisiti**, nella quale i requisiti vengono descritti in modo rigoroso;
4. **convalida dei requisiti**, nella quale si verifica che i requisiti siano completi, consistenti, realizzabili e verificabili.

La convalida usa quattro criteri distinti. Un requisito è *completo* se non lascia lacune rilevanti ed è *consistente* se non contraddice altri requisiti. È *realizzabile* se può essere soddisfatto con tecnologia, tempo e budget disponibili; è *verificabile* se si può controllare che sia stato soddisfatto. La disponibilità delle risorse e la possibilità di eseguire un controllo sono quindi valutazioni diverse.

I requisiti si distinguono in due classi principali:

requisiti funzionali indicano che cosa il sistema deve fare, cioè le funzionalità offerte;

requisiti non funzionali indicano come il sistema deve operare o quali vincoli deve rispettare, per esempio affidabilità, efficienza, prestazioni e manutenibilità.

Requisiti e qualità sono collegati: soddisfare e verificare i requisiti permette di valutare la qualità del prodotto, mentre requisiti scritti male rendono difficile o impossibile verificare la correttezza.

La criticità influenza il rigore richiesto alle specifiche: si pensi a sistemi per somministrare farmaci, guida autonoma e gestione di ingenti transazioni. Più gravi sono gli effetti di un errore, più occorre rendere espliciti vincoli, casi limite e criteri di verifica. Lo studio di fattibilità valuta anche tempo, budget e interesse a realizzare il progetto; durata e condizioni economiche non sono fissate dal metodo.

3.2 Dai requisiti alla struttura del sistema

La progettazione stabilisce come strutturare il software per realizzare le specifiche. Le attività comprendono: suddivisione dei requisiti, identificazione dei sottosistemi o architettura software, specifica delle responsabilità dei sottosistemi, progettazione di interfacce, componenti, strutture dati e algoritmi.

Ogni attività produce un documento o ne completa uno esistente per descrivere un modello: modello degli oggetti, modello di sequenza, modello di transizione degli stati, modello strutturale o modello data-flow.

Per rendere gestibili richieste grandi, il progettista le suddivide in problemi più piccoli e indica quali sottosistemi sono responsabili di ciascun requisito. A livello architetturale definisce componenti e comunicazioni ad alto livello; nella progettazione dettagliata precisa interfacce, algoritmi e strutture dati interne.

La scomposizione deve rispettare coesione e singola responsabilità. Attributi e operazioni vanno raggruppati in classi con uno scopo chiaro, evitando sia classi troppo generiche sia dettagli non necessari. La divisione del lavoro riguarda quindi tanto i requisiti assegnati ai sottosistemi quanto le responsabilità delle classi.

3.3 Casi di uso: attori, confine e obiettivi

Un **diagramma dei casi d'uso** descrive il comportamento del sistema come appare dall'esterno, tramite una sequenza di messaggi fra sistema e attori. Un **attore** è una persona o un sistema esterno che interagisce con il prodotto da sviluppare con un certo ruolo; nei diagrammi è rappresentato come un omino con il nome scritto sotto.

Un **caso d'uso** è una funzionalità offerta dal sistema, disegnata come un'ellissi con il nome all'interno; non descrive il comportamento interno né la struttura interna del programma.

In un caso d'uso l'attore cerca di raggiungere un obiettivo, e lo stesso caso d'uso può avere più *scenari*, cioè percorsi alternativi.

Il rettangolo del sistema delimita il confine: gli attori stanno fuori, i casi d'uso dentro. Una *associazione* fra attore e caso d'uso è una linea che li congiunge. Si distinguono inoltre:

generalizzazione fra attori l'attore più specifico eredita le capacità di quello più generale e ne aggiunge altre;

generalizzazione fra casi d'uso il caso d'uso più specifico eredita da uno più generale e aggiunge caratteristiche;

estensione un caso d'uso aggiunge a un caso base un comportamento incrementale opzionale;

inclusione un caso d'uso incorpora il comportamento di un altro.

Per distinguere `<<include>>` da `<<extend>>` occorre osservare anche il verso della freccia tratteggiata: in `<<include>>` la freccia parte dal caso includente e punta al caso incluso; in `<<extend>>` la freccia parte dal caso estendente e punta al caso base.

Livelli di astrazione

Il livello di dettaglio deve restare adatto alla comunicazione. Si possono distinguere tre livelli di astrazione:

- *kite level*, molto astratto e orientato agli obiettivi di business;
- *sea level*, livello medio e raccomandato perché vicino agli obiettivi dell'utente;
- *fish level*, troppo dettagliato e quindi potenzialmente enorme.

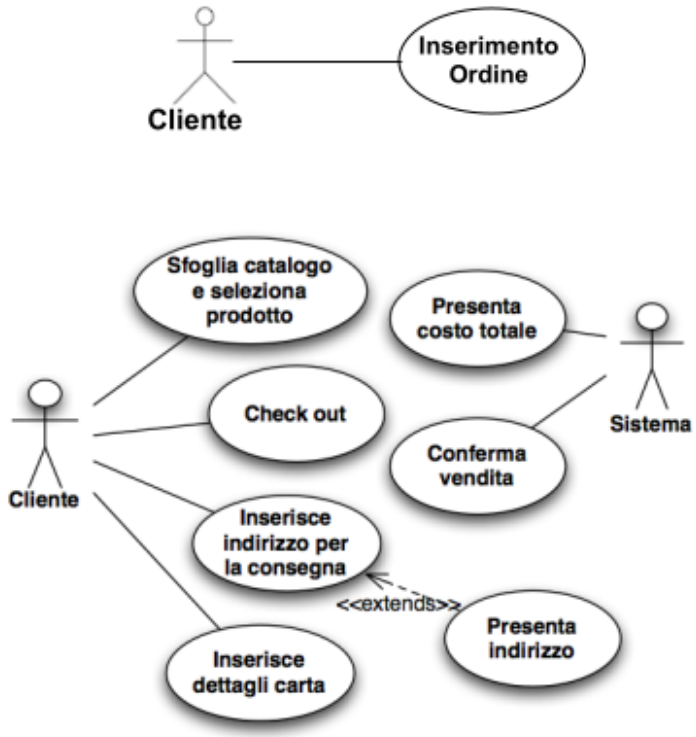


Figura 3.1: Dal caso di uso generale Inserimento Ordine agli obiettivi del cliente e al comportamento opzionale.

3.4 Diagrammi delle attività: decisioni, parallelismo e corsie

Un **diagramma delle attività** rappresenta come si svolge un processo passo per passo. Permette di ragionare sulle operazioni che il sistema dovrà realizzare, prima di scegliere gli oggetti che le svolgeranno. Analizzare il flusso può aiutare a progettare, nel passaggio successivo, le collaborazioni fra oggetti.

La notazione essenziale è:

- rettangoli arrotondati per i passi di elaborazione,
- frecce continue per i flussi,
- rombi per ramificazioni condizionali o merge,
- cerchio pieno per lo stato iniziale e cerchio pieno circondato da una circonferenza per lo stato finale.

Nei diagrammi con flussi paralleli, le barre nere indicano punti di parallelizzazione, quando una freccia entrante si divide in più frecce uscenti, oppure punti di sincronizzazione o *join*, quando più frecce entranti confluiscono in una uscente.

Le *corsie*, o *swimlane*, sono partizioni che indicano il responsabile, attore o modulo, delle attività nella corsia. In questa variante i rettangoli possono rappresentare oggetti o dati, mentre le frecce tratteggiate indicano dati in input o output per un'attività.

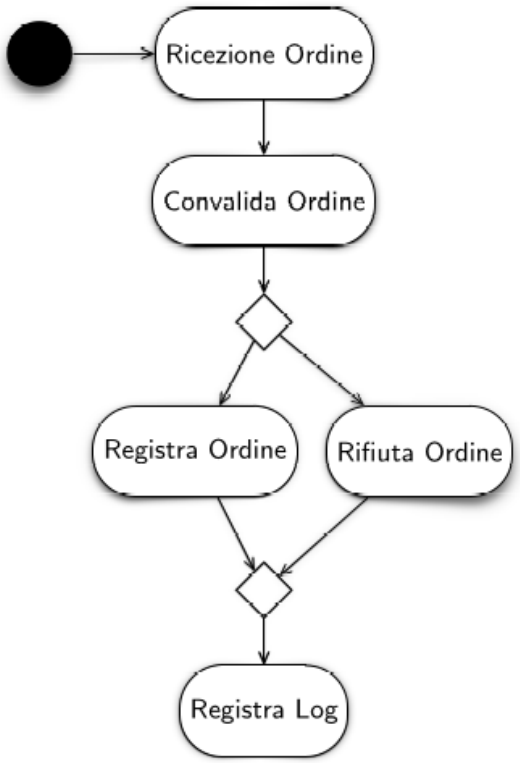


Figura 3.2: Ricezione, convalida, registrazione o rifiuto di un ordine e registrazione del log.

Nel processo di ricezione dell'ordine, il rombo seleziona un'alternativa; il successivo ricongiungimento dei rami permette di passare alla registrazione del log indipendentemente dal ramo

scelto. Un *merge* riunisce alternative, mentre un *join* sincronizza flussi paralleli: non sono la stessa operazione.

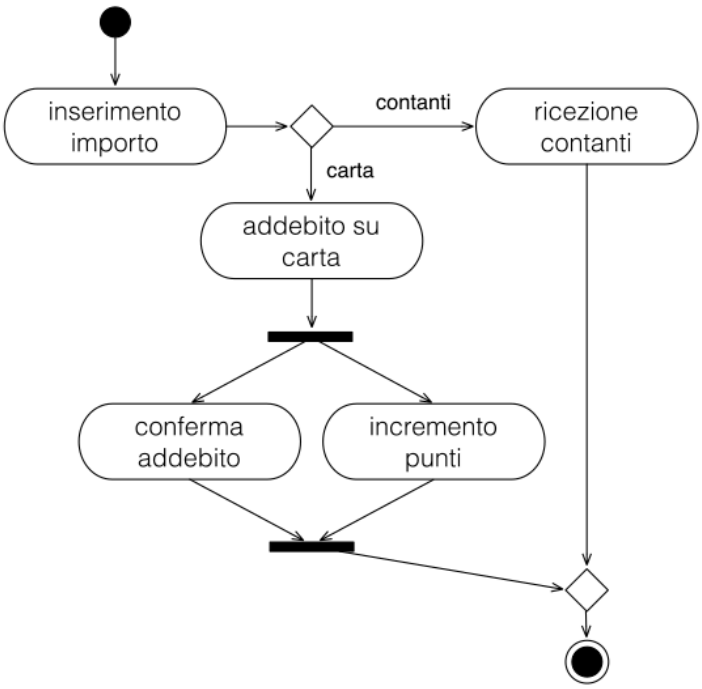


Figura 3.3: Pagamento con alternative e attività concorrenti.

Nel processo di pagamento si separano contanti e carta. Nel ramo carta la barra di fork avvia conferma dell'addebito e incremento dei punti; la barra di join attende i flussi previsti. Una barra può collegare più di due flussi: il numero due riguarda soltanto l'esempio.

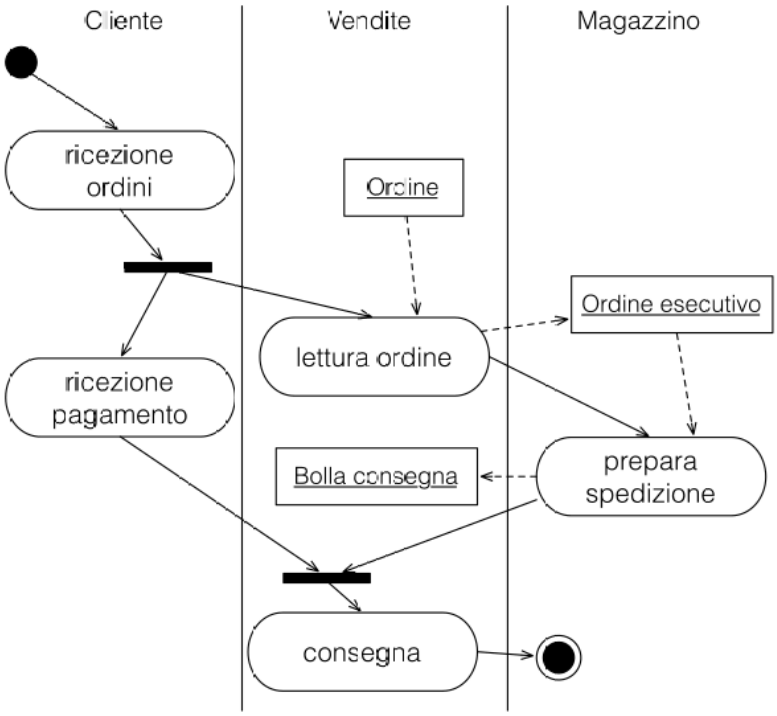


Figura 3.4: Corsie Cliente, Vendite e Magazzino e scambio di documenti.

Le corsie Cliente, Vendite e Magazzino identificano i responsabili delle attività. Ordine e bolla rappresentano invece dati o documenti scambiati fra tali responsabili: distinguerli dalle azioni evita di confondere il contenuto del lavoro con chi lo esegue.

3.5 Diagrammi degli stati: eventi, guardie e azioni

Un **diagramma degli stati** segue un singolo oggetto nel tempo e mostra come il suo stato può cambiare.

Uno **stato** descrive un intervallo della vita dell'oggetto, caratterizzato da valori, dall'attesa di eventi o dallo svolgimento di azioni. La **transizione** descrive invece il passaggio: permette di lasciare uno stato in risposta a un evento e indica una possibile condizione, l'azione eseguita e lo stato di arrivo. Per leggere il diagramma occorre dunque riconoscere sia la situazione in cui si trova l'oggetto sia ciò che può farlo passare alla successiva.

L'etichetta di una transizione può essere letta nella forma:

$$\text{evento}\,[\text{condizione}]/\text{azione}. \tag{3.1}$$

Qui *evento* è ciò che può far scattare il passaggio di stato, *condizione* è una guardia booleana opzionale scritta tra parentesi quadre, e *azione* è l'operazione eseguita durante la transizione. Se l'evento accade ma non riguarda una transizione uscente dallo stato corrente, non accade nulla secondo il modello.

Uno *stato composto* contiene sottostati, mentre uno stato semplice non ne contiene. I sottostati possono essere sequenziali o concorrenti: nel primo caso ne è attivo uno solo in un certo momento;

nel secondo sono paralleli. Quando l'oggetto esce da uno stato composto concorrente, esce da tutti i suoi sottostati.

Le indicazioni `entry`, `do` e `exit` indicano rispettivamente operazioni eseguite entrando nello stato, durante la permanenza nello stato e uscendo dallo stato.

Per distinguere evento, condizione e azione si possono confrontare due etichette:

- `ricevi[totale<25]`, dove `ricevi` è l'evento e `totale<25` è la condizione;
- `approvato/debit()`, dove `approvato` è l'evento e `debit()` è l'azione eseguita durante la transizione.

Precisazione

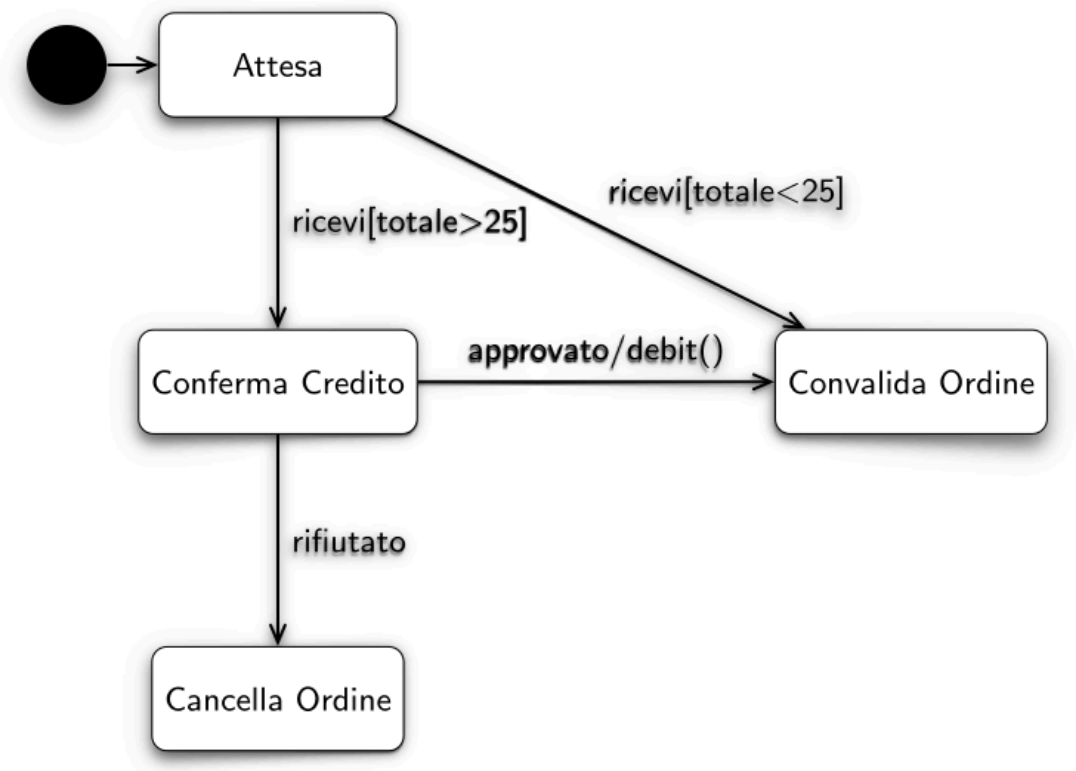


Figura 3.5: Esempio di transizioni con eventi, guardie e azioni.

La guardia va valutata nello stato da cui parte la transizione a cui si riferisce. Il testo dell'esempio presenta `ricevi` con soglie sul totale e un'azione `debit()` associata all'approvazione: evento, condizione e azione hanno ruoli diversi. Le soglie vanno rese complete anche per l'uguaglianza al limite quando si scrive la specifica eseguibile; un diagramma di esempio non sostituisce la decisione sul comportamento al limite.



Figura 3.6: Stato composto sequenziale e concorrente.

L'ingresso in uno stato composto può usare un punto iniziale interno oppure una transizione esplicita a un sottostato; l'uscita può attraversare il confine dello stato composto. Le regioni separate dalla linea tratteggiata indicano concorrenza. L'attività svolta durante la permanenza, indicata con `do` o descritta come *doing*, può essere interrotta da una transizione.

Una notazione esemplificativa delle tre attività è:

```
entry / op1()
do    / op2()
exit  / op3()
```

L'ingresso attiva `op1()`, l'uscita attiva `op3()`, mentre `op2()` descrive l'attività durante la permanenza: una transizione può interromperla. I prefissi appartengono al modello di stato, non sono dichiarazioni Java.

3.6 Diagramma delle classi e relazioni strutturali

Un **diagramma delle classi** rappresenta la struttura statica del sistema. Un rettangolo rappresenta una classe ed è suddiviso in tre sezioni: nome, attributi e metodi. Per attributi e operazioni si usa una sintassi del tipo:

```
- nomeAttributo: Tipo
+ nomeMetodo(param: Tipo): TipoRitorno
```

I simboli di visibilità sono `-` per privato, `+` per pubblico e `#` per protetto. Attributi e metodi sottolineati sono statici, cioè appartengono alla classe e non alle singole istanze. Un'interfaccia può essere rappresentata come un rettangolo con la dicitura `<<interface>>` sopra il nome oppure con la notazione a cerchio, detta *lollipop*, che ne mostra il nome senza elencare le operazioni.

Nei rettangoli delle interfacce usati nel corso si mostrano le operazioni del contratto e si omettono gli attributi. È una scelta di rappresentazione, da distinguere dalle regole Java: un'interfaccia Java non ha campi di istanza, ma può dichiarare campi `public static final`, come spiegato nel §2.4.1.

Le frecce principali sono:

ereditarietà freccia con triangolo vuoto rivolta verso la superclasse; il triangolo deve rimanere vuoto;

implementazione di interfaccia freccia tratteggiata con triangolo vuoto rivolta verso l'interfaccia;

associazione linea continua, eventualmente orientata, che rappresenta una relazione strutturale; può avere molteplicità agli estremi;

dipendenza freccia normalmente tratteggiata che esprime un uso, per esempio di un tipo passato come parametro.

Relazioni parte–tutto e molteplicità

Le relazioni parte–tutto distinguono *aggregazione*, con rombo vuoto, quando una parte può esistere indipendentemente dal tutto, e *composizione*, con rombo pieno, quando una parte appartiene esclusivamente al tutto e ne condivide il ciclo di vita.

Le molteplicità descrivono quante istanze possono essere collegate: zero o più ordini per un cliente e un cliente per ogni ordine, per esempio, esprimono una relazione uno–a–molti. Nei membri di classe la sottolineatura indica `static`; i nomi di classi e operazioni astratte sono in corsivo.

3.6.1 Dalle relazioni ai campi di Ordine

Nel modello della gestione ordini, `ElencoOrdini` raccoglie gli ordini; ciascun `Ordine` conosce cliente, fornitore e prodotti. `Rubrica` organizza i clienti, mentre `ReportMensile` consulta clienti e fornitori per aggregare gli importi. Cliente conserva nome e provincia; Fornitore il nome; Prodotto nome, costo e linea. Questi collegamenti rendono possibili le operazioni individuate dai requisiti.

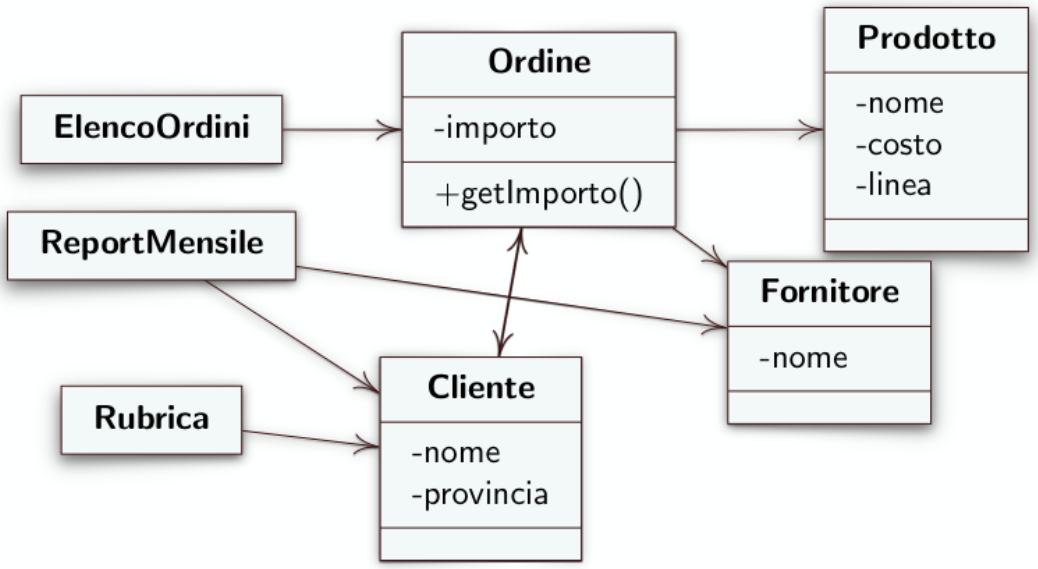


Figura 3.7: Associazioni fra ordini, clienti, prodotti, fornitori e classi di raccolta.

La rappresentazione in Java può esplicitare la corrispondenza fra attributi semplici e riferimenti:

```
class Ordine {
    private int id;
    private java.util.Date data;
    private int importo;
    private boolean archiviato;
    private Cliente cli;
    private Fornitore forn;
    private Stato[] s;
    private Prodotto[] prod;
```

```
    public int getImporto() { return importo; }
    // verifica() richiede le regole applicative di validita'.
}
```

Il listato rappresenta la struttura, non l'intera applicazione: Cliente, Fornitore, Stato e Prodotto sono i tipi del modello. `id`, `data`, `importo` e `archiviato` sono dati semplici; `cli` e `forn` rappresentano associazioni a oggetti; gli array rappresentano riferimenti multipli a stati e prodotti.

L'array, da solo, non specifica tutti i limiti di molteplicità e non impone che gli elementi appartengano esclusivamente al contenitore o ne condividano il ciclo di vita. Se gli stati appartengono esclusivamente all'ordine si può modellare una composizione; un prodotto non acquista quel vincolo soltanto perché è conservato in un campo.

Il corpo di `verifica()` dipende da requisiti di validità che devono essere precisati, mentre `getImporto` è già una query definita.

Leggere il diagramma dettagliato degli ordini

Il diagramma seguente rende espliciti alcuni dati e vincoli lasciati sullo sfondo nel modello generale. `Cliente` conserva `id: int` e `nome: String`, con le operazioni `setNome` e `getNome`; `Fornitore` conserva nome e partita IVA, con `insert(): int` e `getPIva(): String`. `Prodotto` espone il costo mediante `getCosto(): int`, mentre `Stato` conserva `data: Date` ed `evento: String` e dichiara `ritardo(): int`.

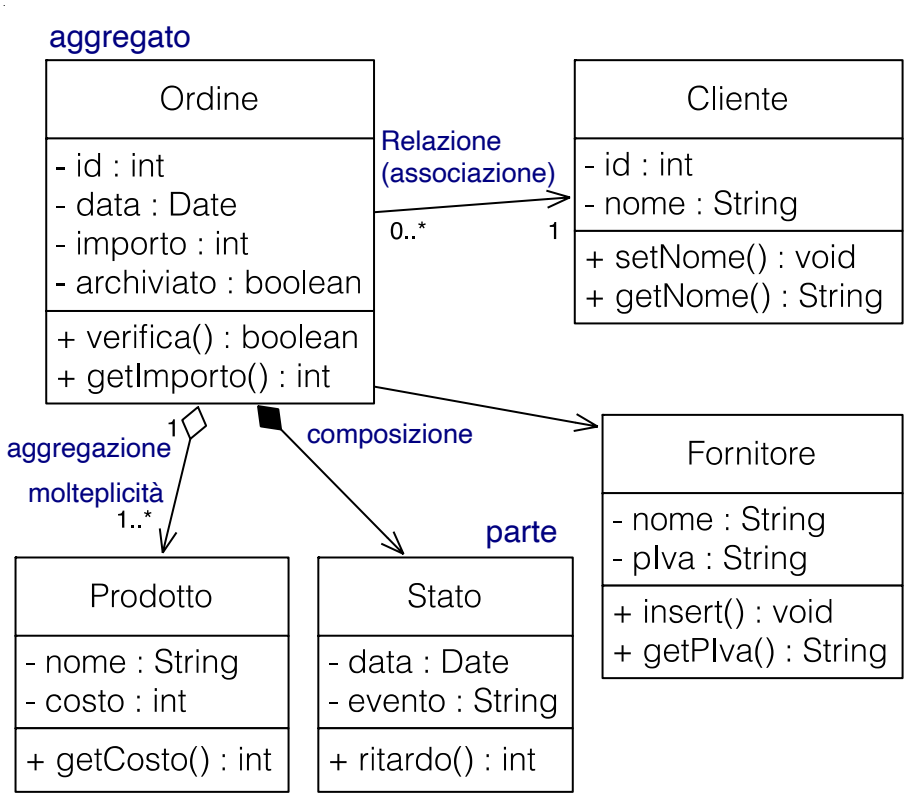


Figura 3.8: Attributi, operazioni e vincoli del modello dettagliato degli ordini.

La relazione con `Cliente` si legge nei due versi: ogni ordine è associato a un cliente, mentre un cliente può avere zero o più ordini, indicati da `0..*`. Nell'aggregazione con `Prodotto`, il rombo vuoto è sul lato dell'ordine; `1..*` sul lato dei prodotti richiede almeno un prodotto per ciascun ordine. Il valore `1` sul lato opposto associa ogni prodotto a un ordine in questo modello. L'aggregazione permette alla parte di esistere indipendentemente dal tutto, ma non implica da sola che la parte sia condivisa fra più ordini: questo dipende dalle molteplicità dichiarate.

Per rappresentare prodotti condivisi da ordini diversi occorrerebbe rivedere tale vincolo o introdurre, per esempio, una distinta riga d'ordine. La composizione con `Stato`, indicata dal rombo pieno, esprime invece che gli stati rappresentati appartengono al ciclo di vita dell'ordine.

La dichiarazione Java `Prodotto[] prod` può rappresentare più riferimenti, ma ammette anche un array vuoto: il vincolo minimo di un prodotto deve essere controllato durante la costruzione e gli aggiornamenti dell'ordine. Le firme del diagramma restano schematiche: `setNome()` non specifica il parametro da assegnare, e per implementare `insert`, `ritardo` o `verifica` servono le rispettive regole applicative.

3.7 Interazioni: collaborazione e sequenza

Un **diagramma di collaborazione** mostra le interazioni fra oggetti. Gli oggetti sono rettangoli identificati dalla notazione `nomeIstanza:nomeClasse`; se il nome dell'istanza non importa si scrive solo `:nomeClasse`. Il flusso di messaggi è indicato da frecce accanto alle associazioni fra le istanze. Le etichette dei messaggi contengono un numero sequenziale, eventualmente gerarchico per chiamate annidate, il metodo chiamato e, se necessario, un valore di ritorno.

Un **diagramma di sequenza** mostra le interazioni fra oggetti durante l'esecuzione. Si legge dall'alto verso il basso, seguendo l'asse temporale; gli oggetti sono invece disposti orizzontalmente.

Per ogni oggetto, la linea di vita (*lifeline*) tratteggiata ne rappresenta l'esistenza. La barra di attivazione indica un periodo in cui l'oggetto esegue un'operazione o attende un ritorno. Una freccia dal chiamante al ricevente rappresenta l'invocazione; la freccia tratteggiata di ritorno può essere omessa. La parola `create` segnala la creazione di un'istanza. Il lettore può così seguire chi chiama, chi esegue e quando l'operazione restituisce il controllo.

I frame `opt`, `alt` e `loop` racchiudono rispettivamente un comportamento opzionale, alternative e una ripetizione.

I diagrammi di collaborazione e di sequenza mostrano aspetti diversi della stessa interazione: la prima evidenzia i collegamenti fra gli oggetti, la seconda l'ordine temporale. Nel diagramma si rappresenta la parte rilevante del comportamento, evitando dettagli che ostacolano la comprensione.

Una chiamata a sé stesso (*self-call*), anche ricorsiva, può tornare sulla stessa linea di vita con un'attivazione annidata. È possibile modellare anche la fine della vita di un oggetto; in Java, tuttavia, il recupero della memoria è gestito dal garbage collector della JVM e non da un distruttore esplicito.

Struttura e singola esecuzione

Diagramma delle classi e diagramma di sequenza descrivono livelli diversi: il primo mostra tipi e relazioni strutturali, il secondo gli oggetti e i messaggi scambiati in una particolare esecuzione. Non è quindi necessario che le frecce dei due diagrammi corrispondano una per una. Un parametro temporaneo, per esempio, può permettere una chiamata senza richiedere un campo persistente nel chiamante.

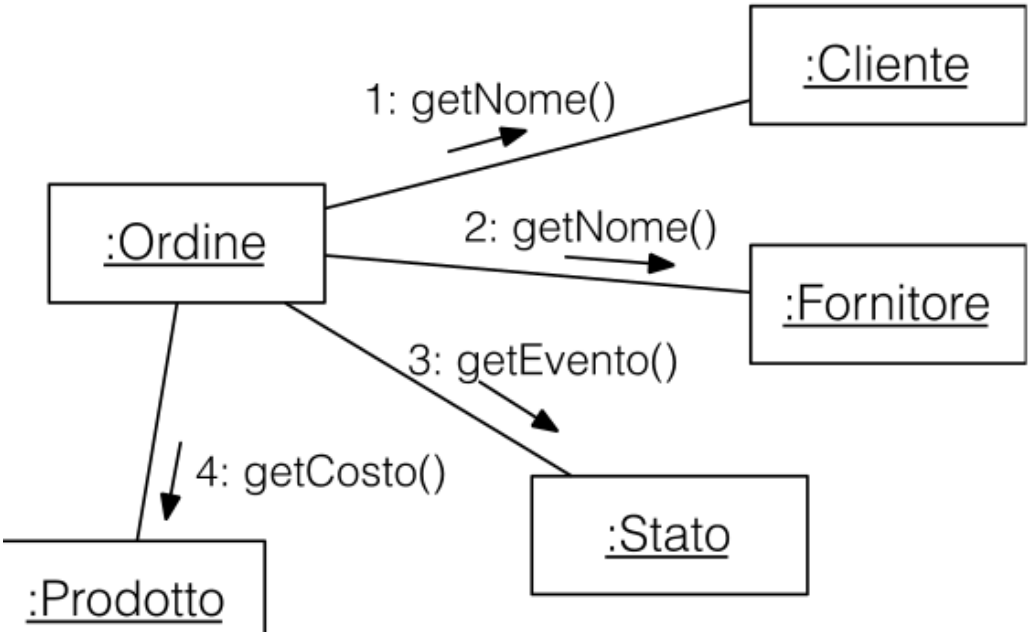


Figura 3.9: Messaggi numerati fra ordine, cliente, fornitore, stato e prodotto.

Nel diagramma di collaborazione gli identificatori delle istanze sono sottolineati. La numerazione rende leggibile l'ordine dei messaggi senza un asse temporale verticale: una notazione gerarchica come `1.1` distingue un messaggio annidato rispetto alla chiamata principale.

Un frame `opt` può racchiudere, per esempio, la creazione di un oggetto richiesta soltanto al primo utilizzo. Il controllo della condizione distingue così un'operazione eseguita sempre da un'inizializzazione differita.

3.7.1 Seguire una sequenza completa: MainEsami

L'esempio Persona–Studente permette di ricostruire tre interazioni collegate. Nel diagramma di creazione MainEsami invoca il costruttore di Studente; la sua inizializzazione richiama il costruttore Persona. Il messaggio `nuovoEsame("Italiano",8)` provoca poi la creazione di un Esame e l'inserimento nella lista; un secondo messaggio registra Fisica con voto 7. La lifeline di un oggetto creato comincia al messaggio di creazione, non prima.

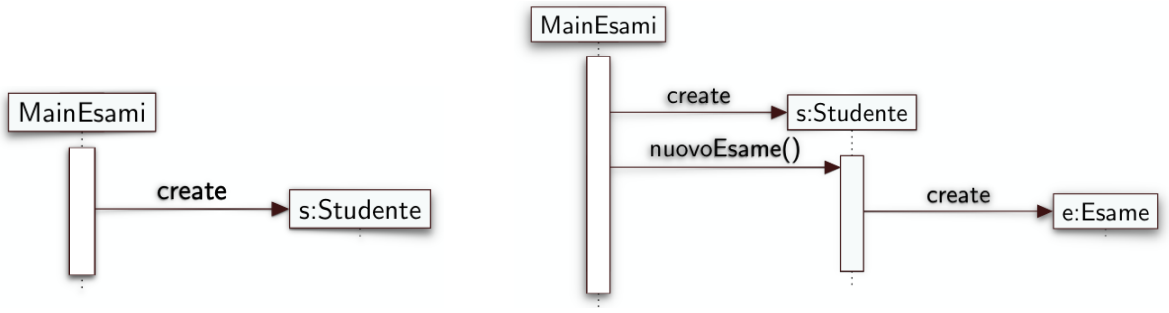


Figura 3.10: Creazione dello Studente e creazione di un Esame attraverso il metodo nuovoEsame.

Nel diagramma di `printAll`, Studente richiama `super.printAll()` per i dati anagrafici e poi stampa matricola ed esami. Il ciclo sugli esami è un frammento `loop`: per ciascun Esame vengono letti materia e voto.

La chiamata interna a `getMedia()` è una chiamata dell'oggetto a sé stesso, rappresentabile con un messaggio che rientra sulla stessa lifeline e un'attivazione annidata. Al suo interno un ulteriore ciclo legge `voto()` per accumulare la somma, poi restituisce $15/2 = 7{,}5$. Le frecce di ritorno riportano valori al chiamante; non rappresentano nuovi oggetti.

Infine MainEsami assegna lo stesso riferimento alla variabile Persona e invoca ancora `printAll`. Non nasce una nuova Persona: la lifeline resta quella dello Studente e il dispatch seleziona ancora la ridefinizione. Il diagramma distingue così costruzione, identità dell'istanza, tipo della variabile e metodi effettivamente eseguiti.

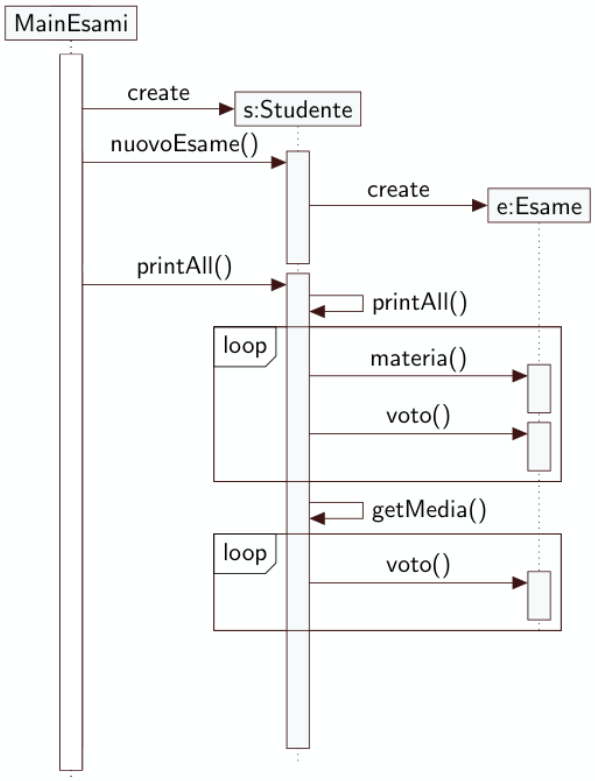


Figura 3.11: Attivazioni, chiamate interne e cicli nella stampa degli esami e nel calcolo della media.

3.8 Individuare classi, attributi e operazioni

L'analisi grammaticale offre un primo procedimento per individuare classi e operazioni nel testo dei requisiti:

1. partire dal testo che descrive i requisiti;
2. individuare i sostantivi: essi suggeriscono classi o attributi;
3. individuare i verbi: essi suggeriscono operazioni o metodi;
4. riconoscere gli oggetti fisici menzionati nel testo come candidati a diventare classi;
5. raggruppare attributi e operazioni nelle classi rispettando coesione, singola responsabilità, KISS e SRP.

Il procedimento è euristico: aiuta a svolgere l'analisi, ma non produce automaticamente il progetto definitivo. Limitarsi alle parole del cliente può rendere il programma fragile, perché il progettista può dover introdurre classi non nominate nei requisiti.

Per distinguere le categorie si possono evidenziare i sostantivi candidati a classi in verde, gli attributi in marrone e i verbi candidati a operazioni in arancione; i colori aiutano a leggere il testo, mentre la scelta di classi, attributi e operazioni dipende da responsabilità e coesione.

Si considerino i seguenti requisiti per clienti, fornitori, ordini, prodotti e report mensili:

- deve essere possibile cercare un cliente e mostrare i suoi dati anagrafici;
- la scheda del cliente deve mostrare tutti i dati anagrafici e una lista dei suoi fornitori;
- deve essere possibile selezionare un intervallo di tempo e calcolare l'importo complessivo degli ordini effettuati da un cliente in tale intervallo;
- per ciascun ordine devono essere mostrati nome fornitore, nome cliente, linea di appartenenza dei prodotti e importo complessivo;
- il report mensile deve mostrare per ciascun cliente la provincia e il totale per ciascun fornitore.

Applichiamo il procedimento al testo. I sostantivi suggeriscono le classi candidate `Cliente`, `Fornitore`, `Ordine`, `Prodotto`, `ReportMensile` e `SchedaCliente`.

Come attributi candidati emergono dati anagrafici, nome cliente, provincia cliente, linea di appartenenza dei prodotti, importo ordine e nome fornitore. I verbi suggeriscono invece le operazioni: cercare un cliente, mostrare dati anagrafici e fornitori, calcolare il totale ordini per cliente, selezionare ordini in un intervallo temporale e calcolare il totale mensile per cliente e fornitore. Sono candidati da valutare in base alle responsabilità, secondo il limite euristico già indicato.